FPT UNIVERSITY

GenCare: Smart Gender Healthcare System

Software Requirement Specification

Class Code: SWR302

Group Code: G3

Ho Chi Minh City, June 30, 2025

RECORD OF CHANGE

*A - Added M - Modified D - Deleted

Effective Date	Changed Items	A/M/D	Change Description	New Version
01/06/2025	SRS baseline	A	Submitted initial skeleton, automatic table of contents and placeholder sections	v0.1
05/06/2025	Process diagrams	A	Added detailed swim-lane diagrams for Appointment, Blog and STI workflows	v0.2
10/06/2025	Authentication chapter	M	Refined login requirements; introduced `email_verified` rule and updated sequence diagram	v0.3
15/06/2025	Sections 2 & 3	M	Completed Overall Description and Technology Stack, aligned with current codebase	v0.4
20/06/2025	Testing strategy	A	Documented Selenium end-to-end suite and Jest unit/integration plan	v0.5
25/06/2025	Performance specification	M	Re-calibrated response-time targets; limited avatar upload to 5 MB	v0.6
27/06/2025	NFR consolidation	M	Merged duplicate NFR chapters; standardised UI stack to Tailwind + shadcn/ui	v0.7
28/06/2025	Road-map features	A	Flagged Manager role, Pill Tracking and Redis rate-limiting as future scope (TBD)	v0.8
30/06/2025	Final QA sign-off	M	Proof-read entire document, fixed formatting, issued release candidate.	v1.0

SIGNATURE PAGE

ORIGINATOR: Tran Phu Tho Date _____

Position: Team Leader

REVIEWERS: Tran Ngoc Nhu Quynh Date _____

Position: Project Reviewer

Contents

RECORD OF CHANGE **1**

SIGNATURE PAGE **2**

1 INTRODUCTION **5**
1.1 Purpose . . . 5
1.2 Definitions, Acronyms . . . 6
1.3 References . . . 7

2 Overall Description **7**
2.1 Product Perspective . . . 7
2.2 Business Process . . . 8
2.2.1 Booking Appointment . . . 8
2.2.2 Booking STI Test . . . 9
2.2.3 Menstrual Cycle and Pill Tracking Management Process . . . 9
2.3 User Classes and Characteristics . . . 10

3 FUNCTIONAL REQUIREMENTS **11**
3.1 Use Case Diagram . . . 11
3.2 Use Case Specifications . . . 11
3.2.1 UC001 - Booking Appointment . . . 11
3.2.2 UC002 - Booking STI Test . . . 15
3.3 State Diagrams . . . 17
3.3.1 STI Test Order State Diagram . . . 17
3.3.2 Result Test State Diagram . . . 18
3.4 Data Flow Diagrams . . . 20
3.4.1 Level 1 DFD - Booking Appointment . . . 20
3.4.2 Level 1 DFD - Booking STI . . . 20
3.5 Logical Data Model . . . 21

4 NON-FUNCTIONAL REQUIREMENTS **23**
4.1 Usability . . . 23
4.1.1 U-1 — User Interface Design . . . 23
4.1.2 U-2 — Accessibility . . . 23
4.1.3 U-3 — Learning Curve . . . 23
4.2 Reliability . . . 23
4.2.1 R-1 — System Availability . . . 23
4.2.2 R-2 — Backup & Recovery . . . 23
4.2.3 R-3 — Error Handling . . . 23
4.3 Performance . . . 23
4.3.1 P-1 — Response Time . . . 23
4.3.2 P-2 — File Upload . . . 23
4.3.3 P-3 — Concurrent Usage . . . 24
4.3.4 P-4 — Video Consultation Quality . . . 24
4.4 Reusability . . . 24
4.4.1 Re-1 — Modular Service Layer . . . 24
4.4.2 Re-2 — Unified Front-End Components . . . 24
4.4.3 Re-3 — API Standardisation . . . 24

4.5 Scalability 24
4.5.1 S-1 — Elastic Application Layer 24
4.5.2 S-2 — Data-Store Growth 24
4.5.3 S-3 — Test Automation Expansion 24

5 SUPPORTING INFORMATION 25
5.1 Testing Framework Implementation 25
5.2 Technology Stack Documentation 26

1 INTRODUCTION

The introduction of the Software Requirements Specification (SRS) provides an overview of the entire SRS. It includes the purpose, scope, definitions, acronyms, abbreviations, references, and overview of the SRS.

The SRS document captures the complete software requirements for the Healthcare Consultation & Testing Platform system. This document captures all requirements using traditional, natural-language style requirements with use-case modeling. It consists of functional and non-functional requirements along with supporting technical specifications.

1.1 Purpose

This Software Requirements Specification (SRS) document specifies the complete requirements for the GenCare - Healthcare Consultation. The SRS fully describes the external behavior of the application and all identified subsystems. It also describes nonfunctional requirements, design constraints, and other factors necessary to provide a complete and comprehensive description of the requirements for the software.

This document is intended for:

- Development team members responsible for system implementation
- Quality assurance team for testing and validation
- Project stakeholders including healthcare professionals and administrators
- System integrators and maintenance personnel
- End users including patients, consultants, and healthcare staff

The GenCare platform is designed to provide:

- Online reproductive health consultation services between patients and certified healthcare consultants
- STI (Sexually Transmitted Infections) testing management and scheduling system
- Personal health record management with menstrual cycle tracking
- Educational blog content delivery for reproductive health awareness
- Multi-role user management system with healthcare-grade security compliance
- Real-time appointment scheduling and reminder system
- Comprehensive reporting and analytics dashboard

1.2 Definitions, Acronyms

This subsection provides the definitions of all terms, acronyms, and abbreviations required to properly interpret the SRS.

API Application Programming Interface

JWT JSON Web Token

STI Sexually Transmitted Infection

SRS Software Requirements Specification

UI User Interface

UX User Experience

RBAC Role-Based Access Control

HIPAA Health Insurance Portability and Accountability Act

OTP One-Time Password

REST Representational State Transfer

CRUD Create, Read, Update, Delete

E2E End-to-End Testing

QA Quality Assurance

CI/CD Continuous Integration/Continuous Deployment

Selenium Web browser automation tool for testing

WebDriver Browser automation API used by Selenium

MongoDB NoSQL document database

React JavaScript library for building user interfaces

Node.js JavaScript runtime environment

Express.js Web application framework for Node.js

Joi JavaScript validation library used for request schema enforcement

Redis In-memory key–value store used for caching, OTP and rate-limiting

Mongoose Object-Document Mapper (ODM) for MongoDB in Node.js

Tailwind CSS Utility-first CSS framework for styling the front-end

shadcn/ui Headless React component library built on Tailwind CSS

OpenAPI Standard, language-agnostic interface description for REST APIs

1.3 References

This subsection provides a complete list of all documents referenced elsewhere in the SRS. Each document is identified by title, report number (if applicable), date, and publishing organization.

1. Circular 49/2018/TT-BYT on Guidelines for Laboratory Testing Activities in Medical Examination and Treatment, Ministry of Health of Vietnam, 28 December 2018
2. Law on Network Information Security No. 86/2015/QH13, National Assembly of Vietnam, 19 November 2015
3. Law No. 15/2023/QH15 on Medical Examination and Treatment, National Assembly of Vietnam, 09 January 2023
4. Decree 13/2023/ND-CP on Personal Data Protection, Government of Vietnam, 17 April 2023
5. Fielding, R. T. — "Architectural Styles and the Design of Network-based Software Architectures" (REST dissertation), University of California, Irvine, 2000
6. Tailwind CSS Utility-first CSS framework
7. shadcn/ui Headless React component library
8. MongoDB Manual Release 7.0, MongoDB Inc., 2024
9. React Documentation (v18), Meta Platforms Inc., 2024
10. Node.js and Express.js Documentation (v18 / v4), OpenJS Foundation, 2024
11. Selenium WebDriver Documentation, SeleniumHQ, 2024
12. Python Testing Best Practices, Python Software Foundation, 2024
13. Jest Testing Framework Documentation (v29), Meta Platforms Inc., 2024

2 Overall Description

2.1 Product Perspective

GenCare is an *all-in-one* tele-healthcare platform built on a client-server architecture:

- **Frontend:** Single-Page Application using React and TypeScript.
- **Backend:** Node.js (Express) with TypeScript, MongoDB for persistent storage, Redis for caching, OTP storage, and rate-limiting.
- **Security:** JWT (access & refresh tokens) and Google OAuth.
- **Third-party integrations:** Gmail SMTP (OTP, notifications) and Google Meet for online consultation rooms.
- **Code organisation:** Controller → Service → Repository → Model, enabling future micro-service extraction.
- **Quality assurance:** Selenium test suites with CI/CD-ready scripts.

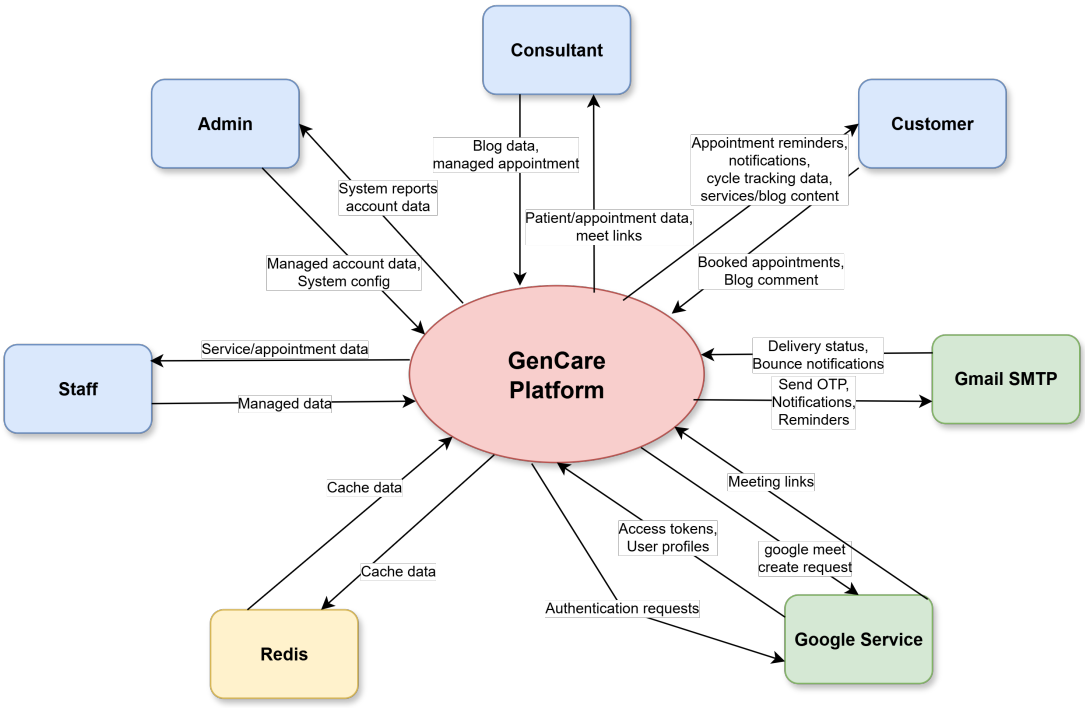


Figure 1: Context Diagram of Gencare system.

Staff & Admin Flow

- **Staff:** clone or adjust STI test schedules, manage STI packages and tests.
- **Admin:** full CRUD of STI packages and tests; audit-log monitoring and system configuration.

2.2 Business Process

The platform supports three primary business flows (swim-lanes):

2.2.1 Booking Appointment

Booking Appointment The appointment booking process is illustrated in Figure 2, which demonstrates the interaction between customers and consultants throughout the booking workflow. As shown in the swimlane diagram, the process begins when a customer selects a preferred consultant and time slot, followed by system validation and consultant confirmation.

Consultant Flow

1. Sign-in and manage personal schedule.
2. Review appointments and access auto-generated Google Meet links.
3. Publish professional blog posts.
4. Monitor feedback ratings.

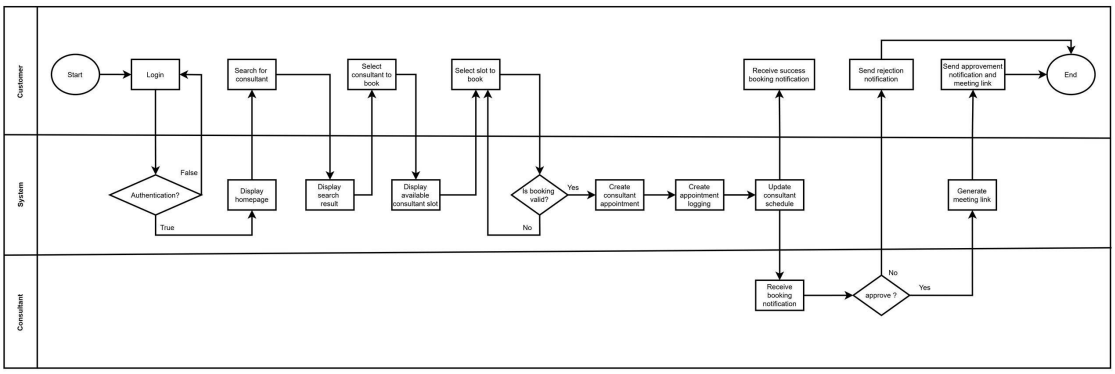


Figure 2: Booking Appointment Swimlane Diagram

Staff & Admin Flow

- **Staff:** clone or adjust consultant schedules, supervise appointments, access statistics.
- **Admin:** full CRUD over users, blogs, audit-log monitoring and system configuration.

All flows trigger notification events through a central `emailNotificationService` and are logged in Redis/MongoDB for reporting.

2.2.2 Booking STI Test

Booking STI Test The STI test booking workflow is detailed in Figure 3, which outlines the complete process from test selection to order confirmation. The diagram shows how customers interact with the system to select appropriate STI test packages and schedule their testing appointments.

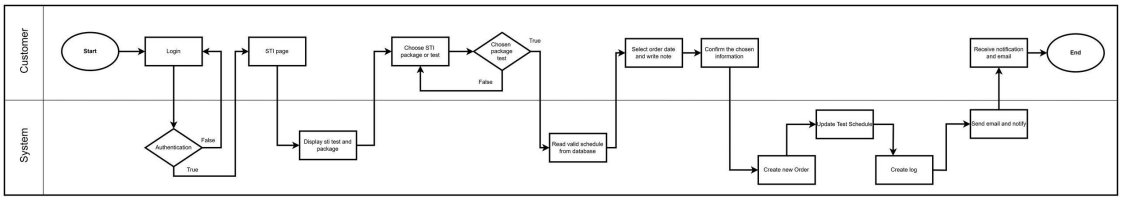


Figure 3: STI Test Booking Swimlane Diagram

Staff & Admin Flow

- **Staff:** clone or adjust STI test schedules, manage STI packages and tests.
- **Admin:** full CRUD of STI packages and tests; audit-log monitoring and system configuration.

2.2.3 Menstrual Cycle and Pill Tracking Management Process

Menstrual Cycle Management The menstrual cycle and pill tracking management process is presented in Figures 4, 5. These diagrams illustrate how the system supports women's health monitoring through automated cycle tracking, prediction algorithms, and medication reminder capabilities.

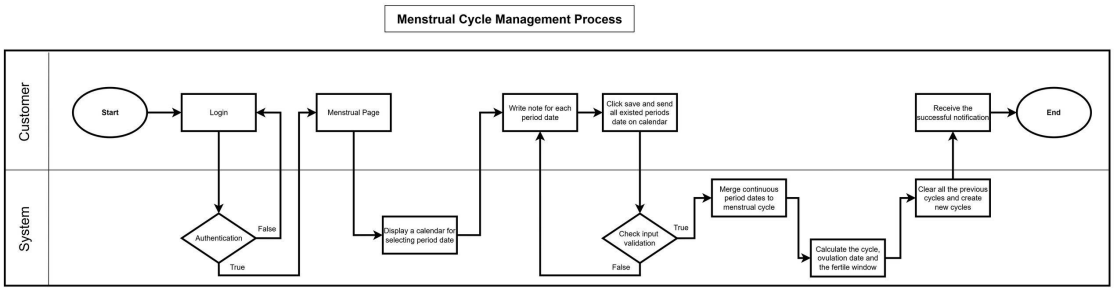


Figure 4: Menstrual Cycle Management Swimlane Diagram

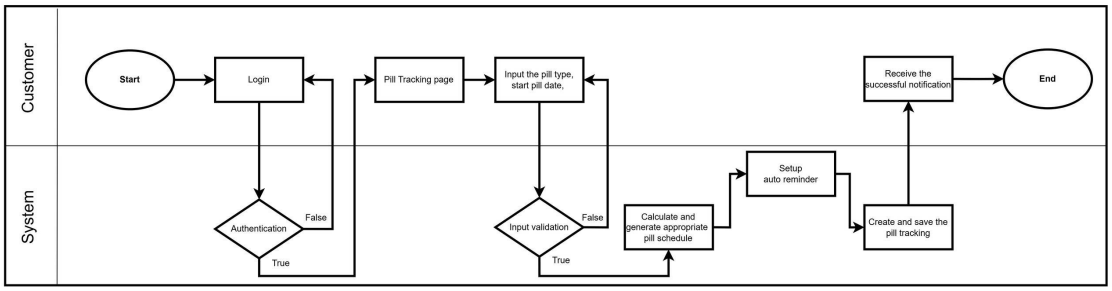


Figure 5: Pill Tracking Management Swimlane Diagram

Health Tracking Process

1. Customer inputs menstrual cycle data and pill-taking schedule.
2. System analyzes patterns and generates predictions for future cycles.
3. Automated reminders are sent for pill-taking and cycle tracking.
4. Healthcare insights and recommendations are provided based on data analysis.

Staff & Admin Flow

- **Staff:** Monitor health tracking compliance and provide support to customers.
- **Admin:** Configure tracking parameters, manage reminder settings, and access aggregate health statistics.

All flows trigger notification events through a central `emailNotificationService` and are logged in Redis/MongoDB for reporting.

2.3 User Classes and Characteristics

Role-based access control is enforced via `authenticateToken` and `authorizeRoles` middleware on the backend.

Table 1: User classes and their main privileges

User Class	Characteristics / Goals	Primary Privileges
Guest	Unauthenticated, anonymous access	View landing page and public blogs
Customer	Patients; mobile-first usage	Sign-up, email verification, booking, cycle tracking, STI packages, feedback
Consultant	Doctors / specialists; busy schedules	Manage timetable, conduct consultations, create blogs, view personal feedback
Staff	Operational support; high-frequency tasks	Coordinate schedules, assist customers & consultants, view statistics
Manager	Strategic oversight; low-frequency usage	Manage staff accounts, approve content, access system-level reports
Admin	Top-level management	Global CRUD on all data, audit-log access, system settings

3 FUNCTIONAL REQUIREMENTS

3.1 Use Case Diagram

The comprehensive functionality of the GenCare Healthcare Platform system is illustrated in Figure 6. This use case diagram demonstrates the various interactions between different user roles and the system's core functionalities.

The use case diagram shown in Figure 6 illustrates the comprehensive functionality of the GenCare Healthcare Platform, showing interactions between different user roles:

- **Unregistered Actor:** Can register, register with Google, and view limited content
- **Registered Actor:** Has access to login, logout, profile management, content viewing, and various healthcare services
- **Customer:** Can book test services, make payments, track test history, view appointment history, book appointments with consultants, join consultations, comment on blogs, rate services, and track menstrual cycles
- **Staff:** Manages test processes, service information, and schedules tests
- **Manager:** Handles staff management (create, update, delete staff) and views reports
- **Consultant:** Posts blogs and manages appointment schedules
- **Admin:** Manages all users and monitors system health and performance

3.2 Use Case Specifications

3.2.1 UC001 - Booking Appointment

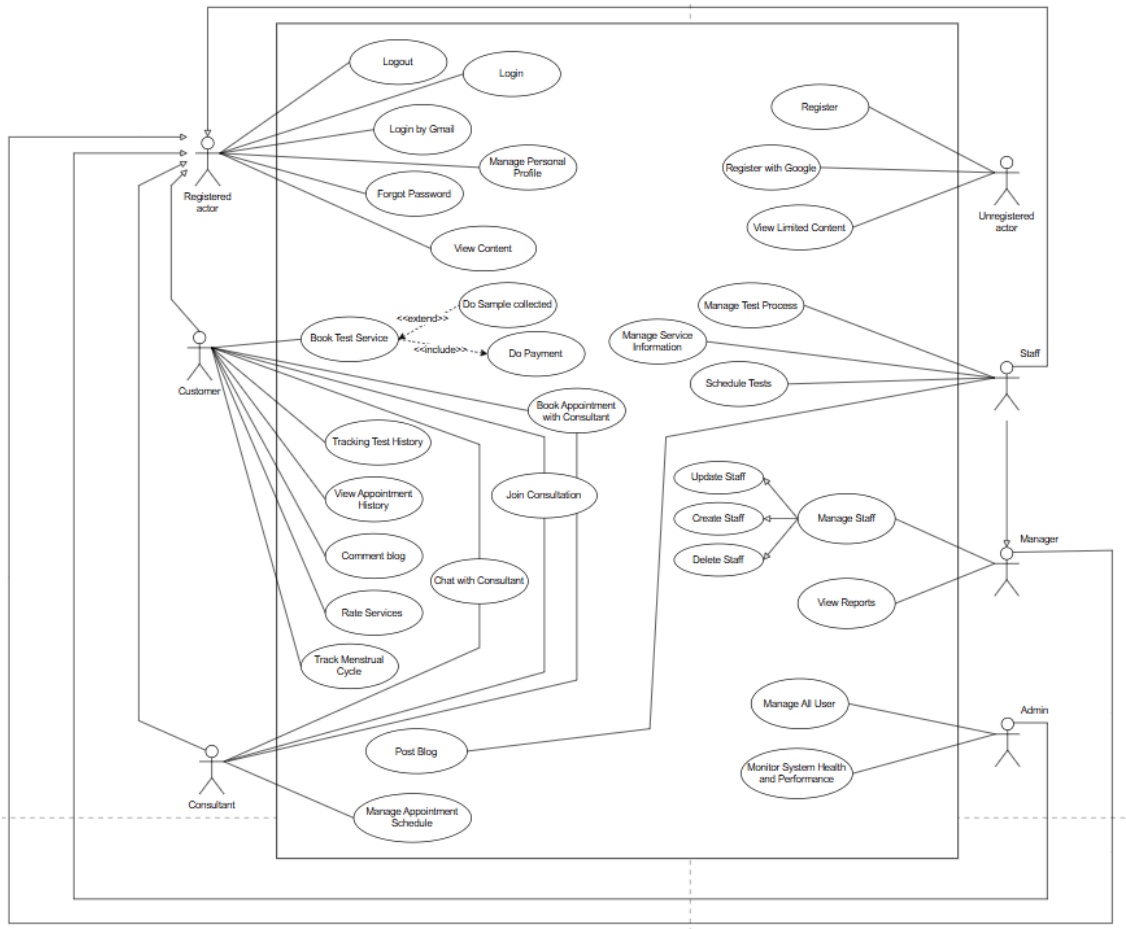


Figure 6: GenCare Healthcare Platform Use Case Diagram

Table 2: Booking Appointment Specification

Field	**Description**
ID and Name	UC-1 Booking Appointment
Created By	Group 3
Date Created	18/06/25
Primary Actor	Customer
Secondary Actors	Consultant
Description	Customer books a consultation appointment with a consultant. The system checks availability, validates business rules, and processes the booking. The appointment is created with pending status awaiting confirmation from the consultant.
Trigger	Customer indicates they want to book an appointment with a specific consultant by clicking booking appointment button.
Preconditions	PRE-1: Customer's identity has been authenticated and logged into the system PRE-2: Customer has a valid account with 'customer' role PRE-3: Customer has no other pending appointments PRE-4: Selected consultant exists in the system

Field	Description
Postconditions	POST-1: Appointment is created with 'pending' status in the system POST-2: Appointment information is recorded in the database POST-3: Appointment history is logged with creation details POST-4: Consultant can view the new appointment in their appointment list POST-5: Customer receives confirmation notification of successful booking
Normal Flow	**3.0 Book Appointment from Consultation System** 1. Customer selects desired consultant 2. System checks customer has no existing pending appointments 3. System validates appointment time must be at least 2 hours in advance 4. System verifies consultant exists and available 5. System retrieves consultant's weekly schedule for the selected date 6. System show consultant's available slots on the selected week 7. System checks appointment time is within working hours and not during break time 8. System creates new appointment with 'pending' status 9. System logs appointment creation in history audit trail 10. System returns success response with appointment details
Alternative Flows	**3.1 Book Appointment from Consultation without login** 1. Before step 1, if customer did not login, navigate to the login page. 2. After login successfully, navigate to step 1 and continue similar to the Normal Flow.

Field	Description
Exceptions	**3.E1 Customer Has Existing Pending Appointment** 1. At step 3, system detects customer already has a pending appointment 2. System displays error message "You already have a pending appointment. Please wait for confirmation or cancel the existing one before booking new appointment." 3. System suggests customer cancel existing appointment or wait for confirmation 4. Use case terminates **3.E2 Insufficient Lead Time** 1. At step 4, system calculates time difference between current time and appointment time 2. If time difference < 2 hours, system returns error with detailed time difference information 3. System suggests customer select alternative time slots 4. Use case terminates **3.E3 Authentication System Unavailable** 1. System detects customer is not authenticated or lacks 'customer' role 2. System returns 401/403 authentication error 3. System redirects to login page and terminates use case **3.E4 Consultant Database Connection Lost** 1. At step 5, system cannot find consultant with provided ID due to database issues 2. System returns error "Consultant not found" 3. System suggests trying again later and terminates use case
Priority	High
Frequency of Use	Approximately 30-60 times per day during business hours
Business Rules	BR-19: Customer can only have 1 pending appointment at a time BR-20: Appointment must be booked at least 2 hours in advance BR-21: Appointment duration minimum 15 minutes, maximum 1 hours BR-22: No overlapping appointments allowed for the same consultant BR-23: Appointments cannot be scheduled during consultant's break time
Other Information	The system must handle timezone processing for online consultations. Integration with weekly schedule system is required for availability verification. Appointment history logging must comply with audit requirements for legal traceability.

Field	Description
Assumptions	Consultants have properly configured weekly schedules. Customers have complete and valid profile information. Stable network connectivity between services.

3.2.2 UC002 - Booking STI Test

The Table 3 illustrates the Booking STI Test process - one of the main flows of our topic. It has all of the fields that is mentioned in the use case specification. Our flow starts with the selection of a specific STI test package from the service list, combined with the assumption that the user has already been authenticated, appropriately authorized as a customer, and that both the STI package and the corresponding test exist within the system. Normally, the service proceeds through the normal flow. However, if the user is not logged in, the process follows an alternative flow. In addition, there are three exceptions, which is described in the field of Exceptions. After finishing the booking, the order will be created with "booked" status and recorded in the database, then write details log for the traceability and accountability.

Table 3: Booking STI Test Specification

Field	Description
ID and Name	UC-02 - Booking STI Test
Created By	Group 3
Date Created	18/06/25
Primary Actor	Customer
Secondary Actors	-
Description	The customer selects STI test packages and a preferred date. System check validation, then confirming the booking, after which the system updates the schedule, sends a confirmation, and logs the order. The STI booking is created with "booked" order status.
Trigger	The customer clicks on a specific STI test package from the service list.
Preconditions	PRE-1: The customer's identity has been authenticated and logged in the system PRE-2: Customer has a valid account with 'customer' role PRE-3: STI packages and tests exist in the system.
Postconditions	POST-1: Order is created with 'booked' status in the system. POST-2: Order information is recorded in the database. POST-3: Order history is logged with details.

Field	Description
Normal Flow	**3.0 Book STI Testing Service** 1. System displays package or tests. 2. Customer selects a package or many tests. 3. System display order date for customer to choose. 4. Customer choose order date. 5. Customer confirm the booking order 6. System update the Test Schedule. 7. System sends confirmed notification. 8. System create logs for storing order history.
Alternative Flows	**3.1 Book STI Testing Service without login** 1. Before step 1, if customer did not login, navigate to the login page. 2. After login successfully, navigate to step 1 and continue similar to the Normal Flow.
Exceptions	**3.E1 No Package or Test Selected** 1. At Step 3, system detects that the customer did not select any package or test. 2. System displays error message: "Please select at least one package or test to proceed." 3. System prompts the customer to return to the selection screen. 4. Use case terminates. **3.E2 Invalid or Past Order Date** 1. At Step 4, system detects that the selected date is invalid or in the past. 2. System returns error: "Selected date is not valid. Please choose a future date." 3. System suggests the customer reselect a valid order date. 4. Use case terminates. **3.E3 Authentication Failed** 1. Before Step 1, system detects that the customer is not authenticated or lacks the 'customer' role. 2. System returns 401 Unauthorized or 403 Forbidden error. 3. System redirects the customer to the login page. 4. Use case terminates.
Priority	High
Frequency of Use	Approximately 20-60 times per day during business hours

Field	Description
Business Rules	BR-30: Customer can choose a package or tests, not both. BR-31: STI tests require patient consent before execution. BR-32: Appointment statuses must follow the defined state flow: • Booked⇒ (Accepted, Canceled) • Accepted ⇒ (Processing, Canceled) • Processing ⇒ SpecimenCollected ⇒ Testing ⇒ Completed. • Canceled and Completed are final and immutable. BR-33: All actions of status change must be logged for legal traceability. BR-34: Canceled orders cannot be reused in any case. BR-35: Each test must be linked to an identified patient and their data must be kept confidential (Law 15/2023/QH15, Vietnam). BR-36: Tests require a valid doctor order or verified self-registration (Circular 49/2018/TT-BYT). BR-37: Patient identity and sample type must be verified before sample collection. BR-38: Test/sample data must not be altered after confirmation. BR-39: Test results must be signed by a certified consultant. BR-40: STI tests for minors require guardian consent. BR-41: Admin actions must be logged for legal traceability and auditing.
Other Information	The system must handle timezone processing for online consultations. Integration with test schedule system is required for availability verification. STI Test history logging must comply with audit requirements for legal traceability.
Assumptions	Customers have valid and authenticated accounts. Available time slots are up-to-date. Logging service is always operational.

3.3 State Diagrams

3.3.1 STI Test Order State Diagram

The STI Test Order State Diagram, presented in Figure 7, illustrates the life cycle of STI test booking within the system. The diagram depicts six distinct states and their corresponding transitions through a unidirectional flow model. As shown in Figure 7, the system begins in the "Available" state, representing time slots ready for customer booking, and transitions to Booked upon successful reservation. From this central state, three potential pathways emerge based on external events: the system may transition to "No-show" when scheduled time expires without customer attendance, to "Canceled" if customers proactively cancel their appointments, or to "In progress" when customers arrive and consultation commences, ultimately reaching "Completed" upon service conclusion. This state machine model is designed to ensure effective handling of all scenarios that may arise during the STI test booking process. The Booked state serves as the central point for determining the next

steps, ensuring accurate tracking and analysis of appointment outcomes.

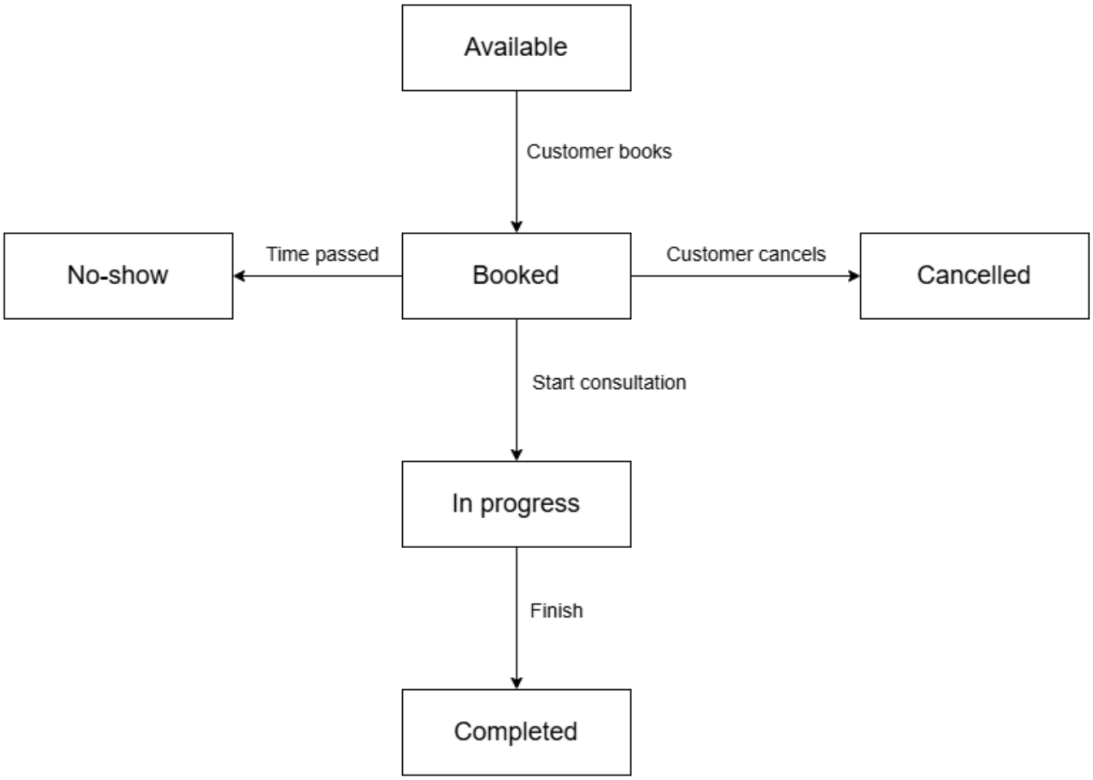


Figure 7: Booking STI Test State Transition Diagram

3.3.2 Result Test State Diagram

The STI test result processing state diagram, illustrated in Figure 8, shows the comprehensive process of handling STI test results from sample collection to final result delivery. The diagram demonstrates eight main states with one branch for handling rejected samples, following a clear and step-by-step flow with built-in quality checks. As depicted in Figure 8, the process starts with "Sample collected", then moves through transportation to the laboratory. At "Received by lab", the system performs sample quality validation. If a sample is not of adequate quality, it is directed to "Sample-rejected" to ensure result accuracy. Acceptable samples continue through "In analysis" and "Analysis complete", followed by a review phase with "Pending review" and "Reviewed", where a qualified healthcare professional checks and approves the results—a critical step for STI testing due to its medical significance and emotional impact. The process concludes with "Reported" for documentation and "Delivered", when results are communicated to the patient. This comprehensive flow ensures accuracy, efficiency, and proper medical review, with appropriate handling of rejected samples.

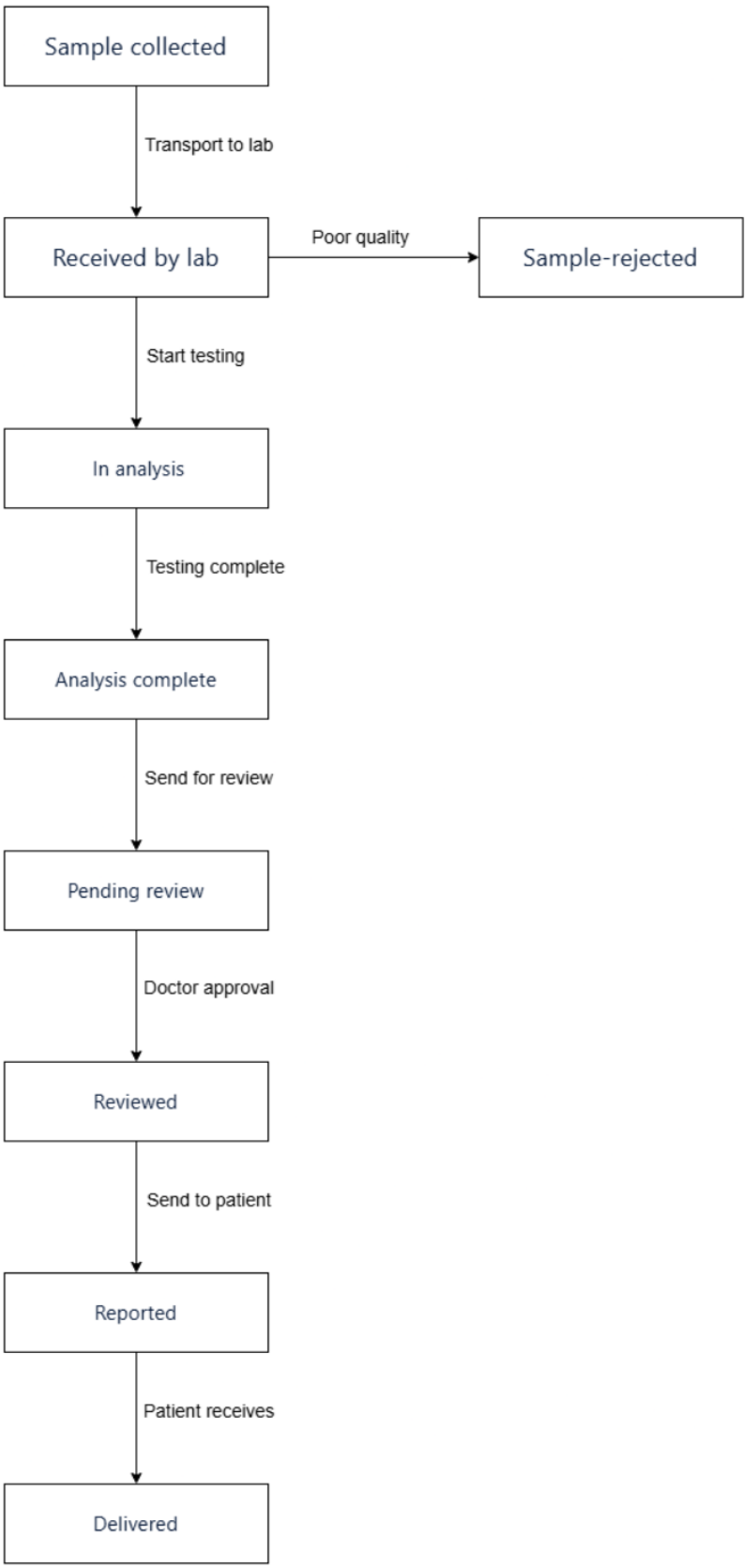


Figure 8: STI Test Result Processing State Transition Diagram

3.4 Data Flow Diagrams

3.4.1 Level 1 DFD - Booking Appointment

The Level-1 Data Flow Diagram for "Booking Appointment" (Figure 9) decomposes the top-level booking process into five distinct subprocesses connected to three internal data stores and two external entities. The primary external actors are the Customer, who initiates all booking interactions, and the Consultant, who ultimately receives the confirmed appointment.

The workflow begins with Process 1 "Validate Request," which authenticates the customer against the Customer Database and either accepts or rejects the booking access request. If validation succeeds, Process 2 "Select Consultant" retrieves a filtered consultant list from the Consultant Database in response to the customer's search criteria. Next, Process 3 "Select Schedule" queries the Weekly Schedule Database to return available time slots for the chosen consultant.

After the customer selects a slot, Process 4 "Book Appointment" writes a new Appointment Record to the Appointment Database and simultaneously creates an audit entry in the History Database. Finally, Process 5 "Confirm/Cancel" allows the customer to confirm or cancel the booking; the system updates appointment status in the Appointment Database, appends a history record, and, if confirmed, forwards the appointment details to the Consultant, who updates his or her weekly schedule accordingly.

As shown in Figure 9, data flows illustrate authentication responses, consultant lists, available times, booking commands, appointment records, and status notifications, ensuring that (a) only authenticated customers can book, (b) consultant schedules remain consistent, and (c) every action is permanently logged for traceability.

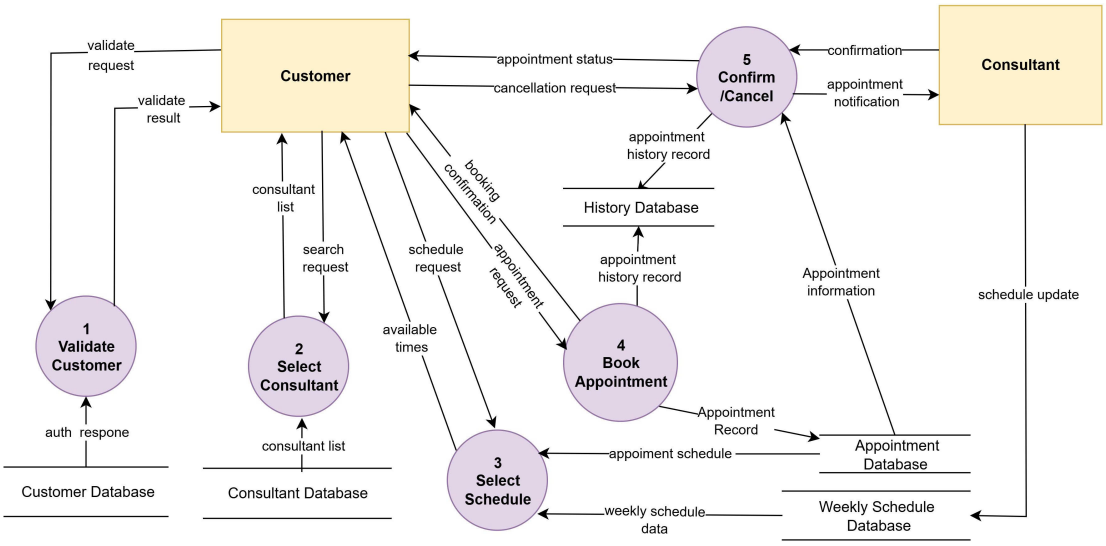


Figure 9: Booking Appointment Data Flow Diagram Level 1

3.4.2 Level 1 DFD - Booking STI

The Level 1 Data Flow Diagram for STI test order processing (Figure 10) illustrates the complete flow of information throughout the STI appointment booking process, highlighting the interactions between users and the system, internal processing steps, and associated databases.

The workflow begins with customer authentication, carried out through the Validate Customer process, which connects to the Customer database. At this stage, the system

checks whether the user has the appropriate role and has successfully logged in. Once validated, the customer proceeds to select a test package, with relevant data retrieved from the STI Package and STI Test databases.

Appointment scheduling is then handled by the Select Order Date process, which accesses information from the STI Test Schedule database. The process concludes with the Create Booking Order step, which generates the order and stores it in the STI Order database, while also recording a log in STI Order History for traceability purposes.

As demonstrated in Figure 10, this design effectively supports the STI test booking workflow from authentication to order creation, while ensuring reliable data management and coordinated processing.

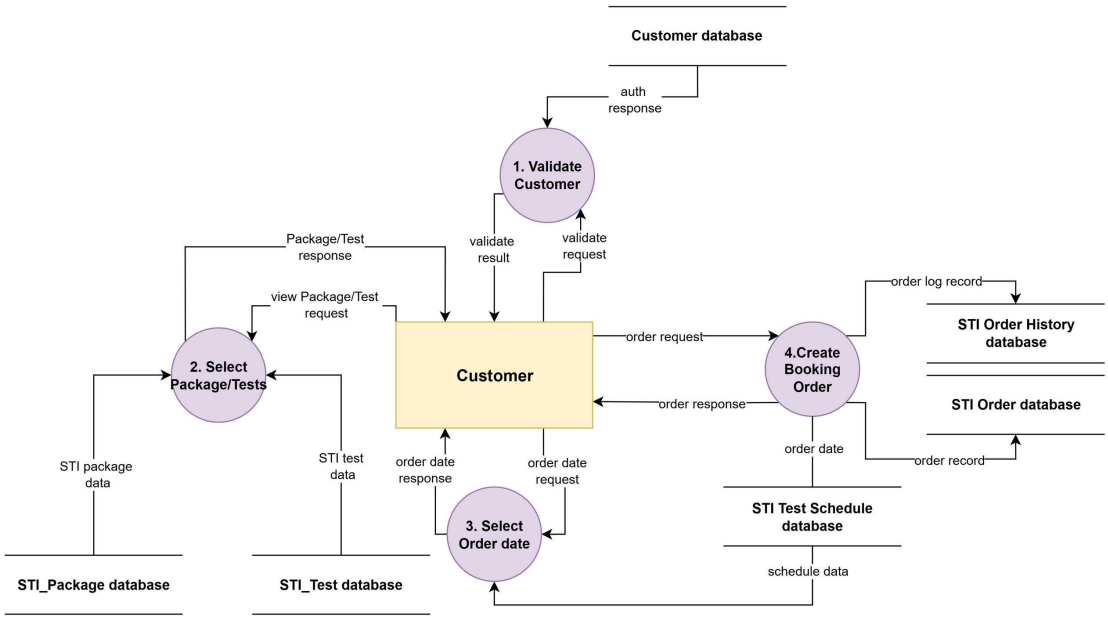


Figure 10: Booking STI Data Flow Diagram Level 1

3.5 Logical Data Model

The logical data model (Figure 11) defines the comprehensive data structure for the healthcare consultation and STI testing management system. The model comprises 17 interconnected entities organized into distinct functional domains as illustrated below:

User Management Domain: The system supports multiple user types through *User*, *Staff*, *Consultant*, and *Customer* entities. The *User* entity serves as the base table containing common authentication and profile information, while specialized entities inherit role-specific attributes. *Staff* manages system operations, *Consultant* provides medical consultations, and *Customer* represents service recipients.

Appointment Management Domain: The core scheduling functionality is implemented through *Appointment*, *AppointmentHistory*, *WeeklySchedule*, and *WorkingDay* entities. The *Appointment* entity manages booking requests and status tracking, while *AppointmentHistory* maintains a complete audit trail of all appointment modifications. *WeeklySchedule* and *WorkingDay* entities define consultant availability patterns and working hours.

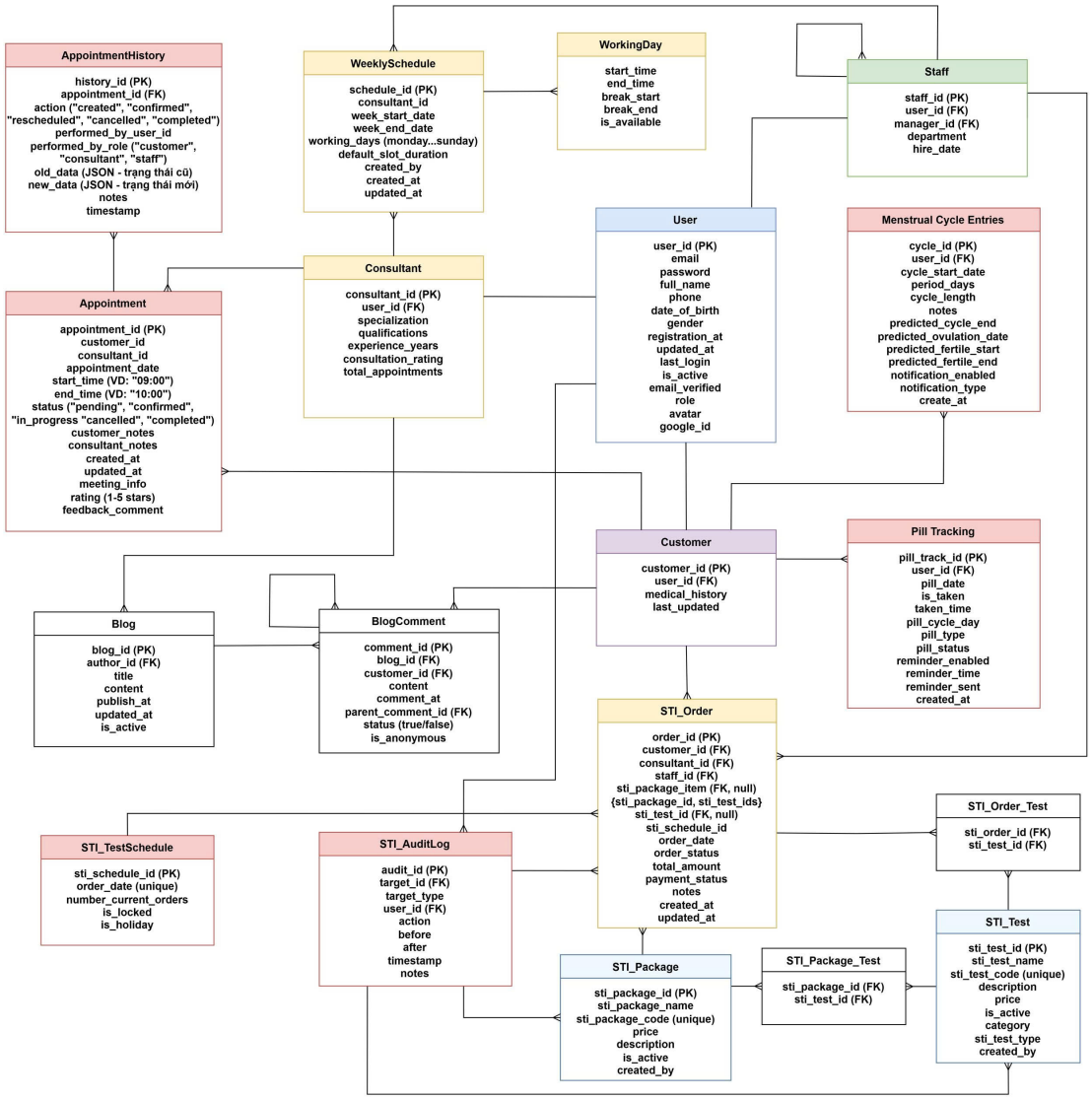


Figure 11: Logical Data Model for Healthcare Consultation & STI Testing System

Healthcare Tracking Domain: Specialized healthcare monitoring is supported through *MenstrualCycleEntries* and *PillTracking* entities. These entities enable personalized women's health monitoring, cycle prediction, and medication adherence tracking with automated reminder capabilities.

STI Testing Domain: The STI testing workflow encompasses *STI_Order*, *STI_Package*, *STI_Test*, *STI_Package_Test*, *STI_Order_Test*, and *STI_TestSchedule* entities. This domain supports flexible test ordering, package bundling, and scheduling. The many-to-many relationships between packages and tests allow for customizable test combinations.

Content Management Domain: Educational content and community engagement are facilitated through *Blog* and *BlogComment* entities, supporting knowledge sharing and user interaction within the healthcare community.

System Audit Domain: The *STI_AuditLog* entity provides comprehensive system activity logging for security monitoring, compliance tracking, and performance analysis.

All entities shown in Figure 11 implement standard temporal attributes (*created_at*, *updated_at*) to ensure data lineage and support regulatory compliance requirements. The model maintains referential integrity through properly defined foreign key relationships and supports both real-time transactional processing and historical data analysis.

4 NON-FUNCTIONAL REQUIREMENTS

4.1 Usability

4.1.1 U-1 — User Interface Design

The system shall provide an intuitive, responsive interface based on a Tailwind CSS design system and the shadcn/ui component library. Primary tasks shall be achievable within three clicks from the main dashboard and navigation shall remain consistent across all modules.

4.1.2 U-2 — Accessibility

The interface shall comply with WCAG 2.1 Level AA: support screen readers, full keyboard navigation and a high-contrast mode. Text shall scale up to 200 % without loss of functionality.

4.1.3 U-3 — Learning Curve

A new user shall be able to register and book a consultation within ten minutes without external training. Context-sensitive help shall be available on every screen.

4.2 Reliability

4.2.1 R-1 — System Availability

The platform shall maintain 99.9 % uptime during operational hours, equivalent to a maximum of 8.77 hours of unscheduled downtime per year. Planned maintenance will be scheduled from 02:00 to 06:00 local time with at least 48 hours' notice.

4.2.2 R-2 — Backup & Recovery

Incremental backups shall run every six hours, with full backups daily and a 30-day retention window. Recovery Point Objective (RPO) is one hour; Recovery Time Objective (RTO) is four hours.

4.2.3 R-3 — Error Handling

All critical errors shall be logged automatically and trigger immediate alerts to administrators. User-facing messages must be clear and actionable while revealing no sensitive information.

4.3 Performance

4.3.1 P-1 — Response Time

Under normal load, 95 % of page navigations must complete in $\leq$ 2 s; REST API calls $\leq$ 500 ms; complex database queries $\leq$ 2s.

4.3.2 P-2 — File Upload

The avatar-upload facility shall accept image files up to 5 MB and confirm upload within 30 s.

4.3.3 P-3 — Concurrent Usage

A single application node shall sustain at least 200 simultaneous sessions without perceptible slowdown. Horizontal scaling will support ≥ 1000 concurrent active users and up to 5000 at peak when auto-scaling is enabled.

4.3.4 P-4 — Video Consultation Quality

Video sessions (powered by Google Meet) shall provide at least 720 p @ 30 fps video and ¡ 150 ms audio latency for up to 100 concurrent meetings; bandwidth optimization is delegated to the provider.

4.4 Reusability

4.4.1 Re-1 — Modular Service Layer

Business rules must be encapsulated in independent TypeScript service modules exposing clear interfaces so that controllers, background jobs or future micro-services can reuse them without modification.

4.4.2 Re-2 — Unified Front-End Components

All screens shall reuse or extend the shared Tailwind / shadcn/ui component set, preserving naming and styling conventions.

4.4.3 Re-3 — API Standardisation

All outward-facing endpoints shall conform to a common response envelope and error pattern. An OpenAPI 3.0 specification will be maintained as the single source of truth.

4.5 Scalability

4.5.1 S-1 — Elastic Application Layer

Because the application is stateless, additional instances can be added or removed behind a load balancer. Autoscaling rules: scale-out at 70 % CPU utilisation, scale-in at 30 %.

4.5.2 S-2 — Data-Store Growth

The current schema and indexing strategy shall support at least 100.000 customers and one million appointment records before sharding or read replicas become necessary.

4.5.3 S-3 — Test Automation Expansion

The Selenium + pytest automation suite shall run tests in parallel; additional browser nodes can be added incrementally to maintain execution time as coverage increases.

5 SUPPORTING INFORMATION

The supporting information makes the SRS easier to use and provides additional context for implementation teams.

5.1 Testing Framework Implementation

Selenium Python Framework Architecture
The automated testing framework shall be implemented using Selenium WebDriver with Python, following the Page Object Model (POM) design pattern for maintainable and scalable test automation:

- **Framework Structure:** Organized into page objects, test cases, utilities, and configuration modules
- **Browser Support:** Chrome, Firefox, Safari, and Edge browsers with configurable driver management
- **Test Data Management:** External test data files in JSON and CSV formats for data-driven testing
- **Reporting:** Detailed HTML reports with screenshots for failed test cases and execution statistics
- **Parallel Execution:** Support for concurrent test execution using pytest-xdist for reduced testing time
- **CI/CD Integration:** Automated test execution in GitHub Actions and Jenkins pipelines

Test Coverage Requirements
The automated test suite shall achieve comprehensive coverage across all system functionality:

- **End-to-End Tests:** Complete user workflows including registration, consultation booking, and test ordering
- **UI Component Tests:** Validation of all user interface elements, forms, and interactive components
- **API Integration Tests:** Verification of all REST API endpoints with various input scenarios
- **Cross-Browser Tests:** Compatibility testing across supported browsers and devices
- **Performance Tests:** Load testing and response time validation for critical user paths

5.2 Technology Stack Documentation

Frontend Technology Stack

- React 18 + TypeScript for type-safe SPA development
- React Context API for client-side state management
- Tailwind CSS with Shadcn UI components for styling and design system
- Axios for HTTP communication with backend REST APIs
- Vite for fast local development and bundling

Backend Technology Stack

- Node.js 18 + Express.js with TypeScript for RESTful services
- MongoDB 5 with Mongoose ODM for data persistence
- Redis for caching, OTP storage and rate-limiting
- JSON Web Token (JWT) authentication with `bcrypt` password hashing
- Joi for request validation and data sanitisation
- Nodemailer for transactional e-mails (OTP, notifications)
- Passport Google OAuth 2.0 strategy for social login
- Nodemon + ts-node for developer hot-reload

Testing and DevOps Stack

- Jest + ts-jest for unit and integration testing of backend functions
- Selenium WebDriver (Python) for end-to-end UI test automation
- Postman for manual API testing and collections storage
- Docker + Docker Compose (Redis service) for local containerisation
- ESLint and Prettier for code quality and formatting